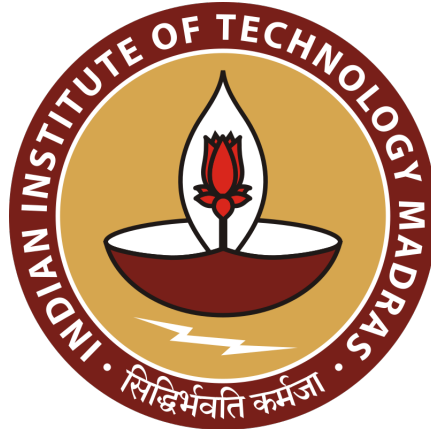


INDIAN INSTITUTE OF TECHNOLOGY, MADRAS



Data Science and AI Project

MILESTONE 3 SUBMISSION

Submitted by:

AMAN KANKRIYA	21F2000493
HARSH KUMAR	22F3002198
SURAJAN DAS	21F1004063
ADITYA NUPANI	23F1000873
SHUBHAM MISHRA	21F3001485

March 19, 2026

Contents

1	Model Architecture and Design	1
1.1	Introduction	1
1.1.1	Reference Architectures	1
1.2	Model Selection	1
1.3	Architecture Overview	1
1.3.1	Input Representation	2
1.3.2	Siamese Encoder	2
1.3.3	ASPP Bottleneck	3
1.3.4	Decoder Network	3
1.3.5	Output Layer	4
1.4	Loss Function	4
1.4.1	Binary Cross Entropy Loss	4
1.4.2	Dice Loss	4
1.4.3	Final Loss Function	4
1.5	Evaluation Metrics	4
1.6	Training Configuration	5
2	End-to-End Pipeline	6
2.1	Implementation Notebook	6
3	Justification of Architecture Choice	7

1 Model Architecture and Design

1.1 Introduction

The objective of this project is to perform pixel-level forest change detection using multi-temporal satellite imagery. Since the task involves identifying spatial changes between two time instances, it is formulated as a **binary semantic segmentation problem**.

To effectively capture both spatial and temporal differences, a deep learning architecture based on a **Siamese U-Net with Atrous Spatial Pyramid Pooling (ASPP)** is selected.

1.1.1 Reference Architectures

The proposed model architecture is inspired by a combination of established deep learning models in semantic segmentation and change detection.

- U-Net Architecture: <https://arxiv.org/abs/1505.04597>
- DeepLab (ASPP Module): <https://arxiv.org/abs/1606.00915>
- Siamese Networks for Change Detection: <https://arxiv.org/abs/1810.08462>

The final model used in this project integrates these concepts into a unified Siamese U-Net architecture with an ASPP bottleneck for multi-scale feature extraction.

1.2 Model Selection

The chosen model architecture is a **Siamese U-Net integrated with ASPP**. This architecture is specifically designed to handle:

- Multi-channel satellite imagery
- Temporal change detection (bi-temporal inputs)
- Pixel-wise classification

The model takes as input two satellite images from different time periods (2022 and 2023) and predicts a binary mask indicating forest loss.

1.3 Architecture Overview

The overall architecture consists of three major components:

- Siamese Encoder
- ASPP Bottleneck
- Decoder with Skip Connections

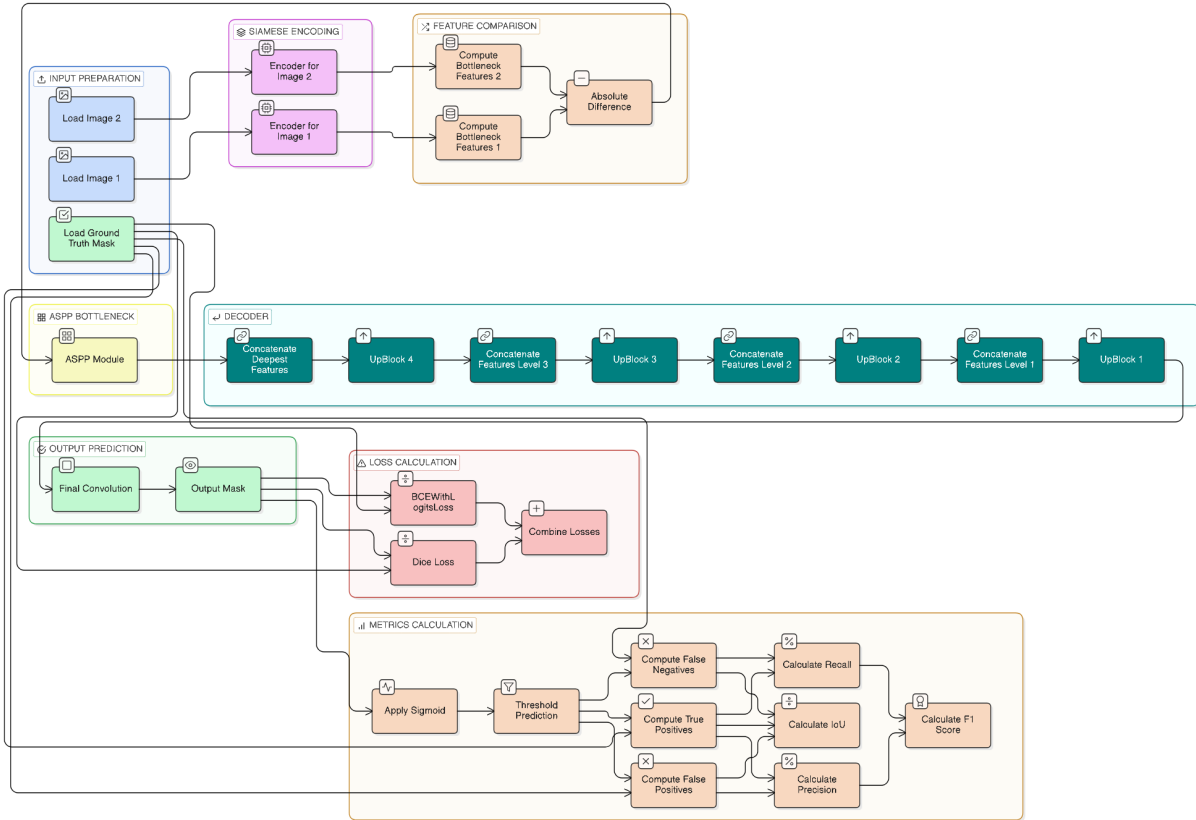


Figure 1: Model Architecture

1.3.1 Input Representation

Each input sample consists of two tensors:

- t_1 : Satellite image from Year 2022 ($6 \times 256 \times 256$)
- t_2 : Satellite image from Year 2023 ($6 \times 256 \times 256$)

These inputs are processed using a shared encoder network.

1.3.2 Siamese Encoder

The encoder follows a Siamese structure, where both temporal inputs (t_1 and t_2) are passed through the same convolutional network with shared weights.

Each encoder block consists of:

- Convolution layer

- Batch Normalization
- ReLU activation

The encoder progressively extracts hierarchical features:

- Low-level features: edges and textures
- Mid-level features: vegetation patterns
- High-level features: forest structure

Using a shared encoder ensures that both temporal inputs are encoded in the same feature space, enabling effective comparison.

1.3.3 ASPP Bottleneck

At the deepest layer of the network, an **Atrous Spatial Pyramid Pooling (ASPP)** module is applied.

ASPP uses multiple convolution layers with different dilation rates to capture multi-scale contextual information.

This is particularly useful for satellite imagery because:

- Forest loss can occur at different spatial scales
- Contextual information improves detection accuracy

The ASPP module enhances the receptive field without increasing computational cost significantly.

1.3.4 Decoder Network

The decoder reconstructs the spatial resolution of the feature maps to generate a segmentation mask.

Key components include:

- Transposed convolution (upsampling)
- Skip connections from encoder layers
- Convolutional refinement blocks

Skip connections help preserve spatial details lost during downsampling, which is critical for accurate boundary detection.

1.3.5 *Output Layer*

The final layer produces a single-channel output:

- Output size: $1 \times 256 \times 256$
- Activation: Sigmoid (applied during inference)

Each pixel represents the probability of forest loss.

1.4 **Loss Function**

The model uses a hybrid loss function combining:

- Binary Cross Entropy (BCE) Loss
- Dice Loss

1.4.1 *Binary Cross Entropy Loss*

BCE loss measures pixel-wise classification error. A weighted BCE loss is used to handle class imbalance, since forest loss pixels are relatively sparse.

1.4.2 *Dice Loss*

Dice loss focuses on overlap between predicted and ground truth masks. It is particularly effective for segmentation tasks with imbalanced classes.

1.4.3 *Final Loss Function*

The final loss is defined as:

$$\text{Loss} = 0.5 \times \text{BCE} + 0.5 \times \text{Dice}$$

This combination ensures both pixel-level accuracy and region-level consistency.

1.5 **Evaluation Metrics**

To evaluate model performance, the following metrics are used:

- Precision
- Recall
- F1-score

Predictions are thresholded at 0.3 to convert probabilities into binary masks.

1.6 Training Configuration

The model is trained using the following setup:

- Optimizer: Adam
- Learning Rate: 1×10^{-4}
- Batch Size: 16
- Number of Epochs: 50
- Hardware: GPU (CUDA-enabled)

To improve training efficiency, mixed precision training is used.

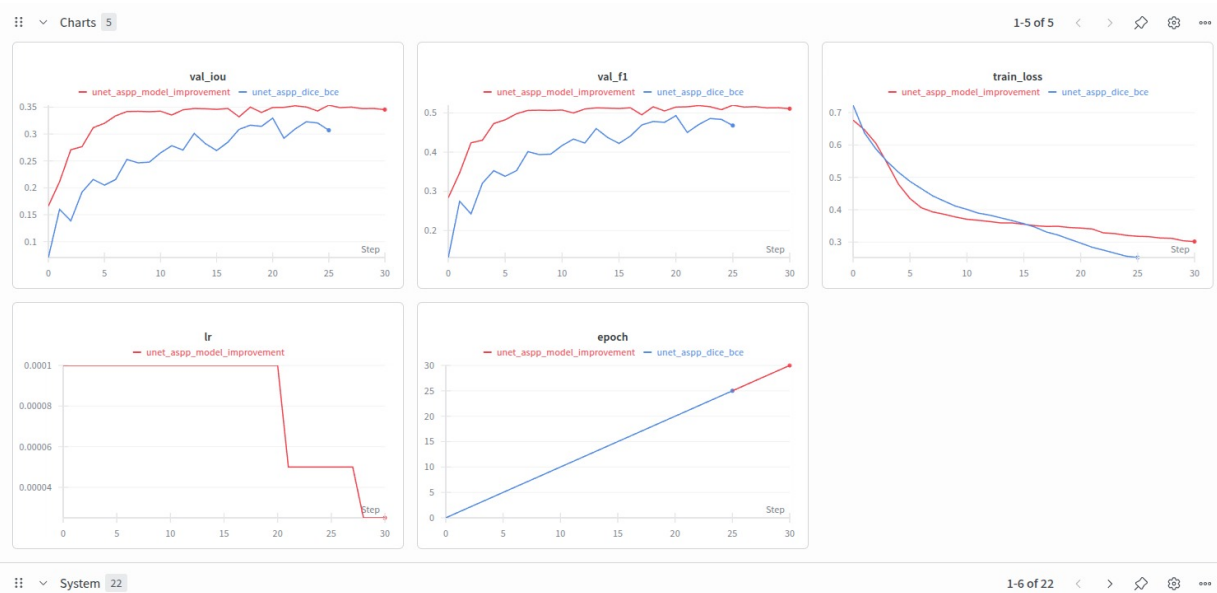


Figure 2: Training Parameters

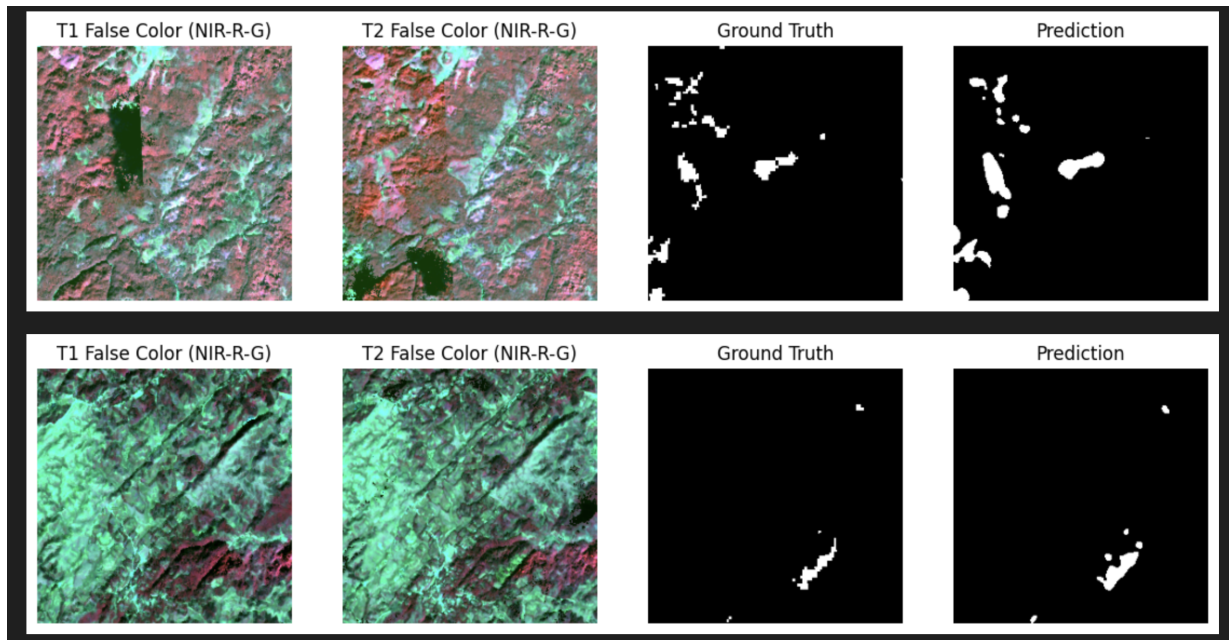


Figure 3: Training Parameters

2 End-to-End Pipeline

The complete pipeline from data to prediction is as follows:

1. **Data Collection** Satellite data is collected from Sentinel-1 and Sentinel-2, and labels are generated from the Hansen dataset.
2. **Preprocessing** Data is normalized and divided into patches of size 256×256 .
3. **Dataset Creation** Bi-temporal image pairs (t_1, t_2) are created along with corresponding labels.
4. **Model Input** The two temporal images are passed into the Siamese encoder.
5. **Feature Extraction** The encoder extracts hierarchical features from both inputs.
6. **Feature Fusion** Features are combined and processed through the ASPP module.
7. **Decoding** The decoder reconstructs the segmentation map.
8. **Prediction** The model outputs a binary mask indicating forest loss.
9. **Evaluation** Predictions are compared with ground truth using evaluation metrics.

2.1 Implementation Notebook

The complete implementation of the model architecture, training pipeline, and experiments is available in the following Kaggle notebook:

- Kaggle Notebook: [Forest Cover Change Detection Model \(Kaggle\)](#)

This notebook includes data loading, preprocessing, model definition, training, and evaluation steps.

3 Justification of Architecture Choice

The selected architecture is well-suited for the problem due to the following reasons:

- **Siamese Structure** Enables effective comparison between two temporal images, which is essential for change detection tasks.
- **U-Net Backbone** Proven performance in segmentation tasks, especially with limited data.
- **ASPP Module** Captures multi-scale contextual information, which is critical for detecting varying sizes of forest loss.
- **Skip Connections** Preserve spatial details, improving segmentation accuracy.
- **Hybrid Loss Function** Addresses class imbalance and improves segmentation quality.

Overall, the architecture provides a balance between performance, interpretability, and computational efficiency, making it suitable for large-scale forest monitoring applications.